

UQFF C++ Source Module

Lines: 1,086 Size: 60,987 bytes

[illegible]

[illegible]

[illegible]

```

190         case SYS_ANYONS_CERN:    return M_any;
191         case SYS_H_RES_FE56:     return M_fe56;
192         case SYS_VAC_HARMONICS:  return M_vac;
193         default:                  return 0.0;
194     }
195 }
196
197 double get_r(int idx) const {
198     switch (idx) {
199         case SYS_W2_TRIADIC:      return r_w2t;
200         case SYS_SGRA_BUOY:       return r_sgra_b;
201         case SYS_PILLARS_DPM:     return r_pil_d;
202         case SYS_RESONANCE_26D:   return r_res26;
203         case SYS_UTE2_SC:         return r_ute2;
204         case SYS_ANYONS_CERN:     return r_any;
205         case SYS_H_RES_FE56:     return r_fe56;
206         case SYS_VAC_HARMONICS:   return r_vac;
207         default:                  return 1.0;
208     }
209 }
210
211 double get_DPM_res(int idx) const {
212     switch (idx) {
213         case SYS_W2_TRIADIC:      return DPM_res_w2;
214         case SYS_SGRA_BUOY:       return DPM_res_sgra;
215         case SYS_PILLARS_DPM:     return DPM_res_pil;
216         default:                  return 1.0;    // other systems: unscaled
217     }
218 }
219
220 /// Effective magnetic field per system (for U_m computation)
221
222 double get_B_eff(int idx) const {
223     switch (idx) {
224         case SYS_W2_TRIADIC:      return 1.0e-5;    // T typical OB-wind ISM
225         case SYS_SGRA_BUOY:       return 1.0e-3;    // T Galactic Centre
226         case SYS_PILLARS_DPM:     return 1.0e-5;    // T ISM pillar
227         case SYS_RESONANCE_26D:   return 1.0e-5;    // T reference
228         case SYS_UTE2_SC:         return B_ute2;    // T UTe2 lab field (16 T)
229         case SYS_ANYONS_CERN:     return 1.0;       // T experiment bench
230         case SYS_H_RES_FE56:     return 30.0;       // T hyperfine at Fe nucleus
231         case SYS_VAC_HARMONICS:   return 1.0e-9;    // T IGM background
232         default:                  return 1.0e-5;
233     }
234 }
235
236 const char* system_name(int idx) const {
237     static const char* names[NUM_SYSTEMS_002] = {
238         &quot;Westerlund 2 [DPM Triadic]&quot;;,
239         &quot;Sgr A* [Buoyancy Dominant]&quot;;,
240         &quot;Pillars of Creation [DPM Resonance]&quot;;,
241         &quot;26D Resonance Layer Framework&quot;;,
242         &quot;UTe2 Topological Superconductor&quot;;,
243         &quot;Anyon Condensate [CERN 2025]&quot;;,
244         &quot;Fe-56 Nucleus [H_res Shell Model]&quot;;,
245         &quot;Cosmic Void [Vacuum Harmonics]&quot;;

```

```

245     };
246     return (idx >= 0 && idx < NUM_SYSTEMS_002) ? names[idx] : &quot;Unknown&quot;
247 }
248
249 const char* regime_label(int idx) const {
250     static const char* labels[NUM_SYSTEMS_002] = {
251         &quot;OB Star Cluster (6 pc, 30 km\&times;98\&times;89)&quot;;,
252         &quot;SMBH (1.27\&times;9710\&times;2\&times;9\&times;10\&times;10 m, 4\&times;9710\&times;2\&times;81\&times;10\&times;10 M\&times;98\&times;89)&quot;;,
253         &quot;Star-Forming Pillar (5 pc, ~10\&times;2\&times;81\&times;10\&times;10 M\&times;98\&times;89)&quot;;,
254         &quot;26 Orthogonal Vacuum States (Solar ref.)&quot;;,
255         &quot;Condensed Matter – 1 nm crystal lattice&quot;;,
256         &quot;Quasiparticle – fm scale (nuclear)&quot;;,
257         &quot;Atomic Nucleus – 4.35 fm (Fe-56)&quot;;,
258         &quot;Cosmic Void – 1 Mpc scale&quot;;
259     };
260     return (idx >= 0 && idx < NUM_SYSTEMS_002) ? labels[idx] : &quot;Unknown&quot;
261 }
262
263 const char* unique_physics_str(int idx) const {
264     static const char* phys[NUM_SYSTEMS_002] = {
265         &quot;DPM integral F_U_Bi_i = int[LENR+DPM_mom+DPM_grav+...] dx ~2.11e208 N; &quot;;
266         &quot;DPM_res=1.67e3; F_LEN=1.56e36 N; x2=-1.35e172 m (quadratic); T_sf=2 Myr.&quot;;,
267
268         &quot;F_U_Bi_i~-8.31e211 N (F_LEN dominant); DPM_res=1.67e7; NSC tidal; &quot;;
269         &quot;Kerr spin correction a*=0.3; QPO 20-min; GW back-reaction dOmega/dt.&quot;;,
270
271         &quot;DPM_res=1.67e7 (UV-enhanced); F_U_Bi_i~2.11e212 N; triadic F_Ubi=9.79e-33 N; &quot;;
272         &quot;f_z,CGM~1.46e-73 ([SSq]-corrected); JWST NIRC&times;2022 pillar density.&quot;;,
273
274         &quot;R(t)=sum_{i=1}^{26} R_i*cos(omega_i*t); R_i=F_base*(1+SSq)*exp(-SSq*i/26); &quot;;
275         &quot;omega_i=2pi/T_sf*i*(1+SSq); [SSq]=0.57; 26 orthogonal vacuum quantum states.&quot;;
276
277         &quot;delta_{n,UTe2}=(2pi)^{n/6}*exp(-SSq*n/26)*(1+f_topo)*exp(-pi); &quot;;
278         &quot;B_threshold=16 T; f_topo=0.20; Andreev STM; non-unitary TQFT; n=1..9.&quot;;,
279
280         &quot;F_UBii,anyons=-F_rel*(E_any/E_LEP)*Q_wave*g*exp(-delta_c^2/(2*sigma^2)); &quot;;
281         &quot;delta_c=1.686; sigma=1.0; Ising braiding E_any=1 MeV; 75% CERN 2025 alignment.&quot;;
282
283         &quot;H_res=A_res*sin(2pi*f_res*t)+U_dp*SC_m*k_nuc+S_shell; Z=26,A=56,N=30; &quot;;
284         &quot;S_shell=0.20 (near magic 28); Fe-56 binding 8.79 MeV/nucleon (global peak).&quot;;
285
286         &quot;U_g2=sum H_m*(1-exp(-SSq*m))*cos(omega*t_n); H_m=sum(1/k)*f_Ub; &quot;;
287         &quot;V_series=sum(1/n^26)*SSq^n~0.570=Li26(SSq); dynamic SSq(n,t).&quot;;
288     };
289     return (idx >= 0 && idx < NUM_SYSTEMS_002) ? phys[idx] : &quot;&quot;;
290 }
291
292 void get_ext(int idx, double& M_ext, double& r_ext) const {
293     switch (idx) {
294         case SYS_W2_TRIADIC: M_ext = M_ext_w2t; r_ext = r_ext_w2t; break;
295         case SYS_SGRA_BUOY: M_ext = M_ext_sgra_b; r_ext = r_ext_sgra_b; break;
296         case SYS_PILLARS_DPM: M_ext = M_ext_pil_d; r_ext = r_ext_pil_d; break;
297         case SYS_RESONANCE_26D: M_ext = M_ext_res26; r_ext = r_ext_res26; break;
298         case SYS_UTE2_SC: M_ext = M_ext_ute2; r_ext = r_ext_ute2; break;
299         case SYS_ANYONS_CERN: M_ext = M_ext_any; r_ext = r_ext_any; break;

```


[illegible]

```

410 }
411
412 ///  $g_{\text{base}} = G \cdot M / r^2$  for system idx.
413 double compute_g_base(int idx) const {
414     return g_base_for(idx);
415 }
416
417 /// Tier-1 buoyancy:  $U_{\text{bi}} = \frac{1}{2} \times g_{\text{base}}$ .
418 double compute_Ubi(int idx) const {
419     return 0.5 * g_base_for(idx);
420 }
421
422 /// Tier-2  $F_{\text{UBii}}$ : standard UQFF oscillatory coupling, scaled by DPM_res.
423 /// For PhD systems using specialised methods (26D, UTe2, anyons, H_res,
424 /// vacuum), this provides the standardised comparison baseline.
425 double compute_F_UBii(int idx, double t) const {
426     double g = g_base_for(idx);
427     double M = get_M(idx);
428     double r = get_r(idx);
429     double dpm = get_DPM_res(idx); // 1.0 for systems [3]–[7]
430     return -beta_i * g * omega_g * (M / r) * U_UA
431         * std::cos(UQFF_LA002_PI * t) * dpm;
432 }
433
434 /// Tier-3: external-body tidal buoyancy  $F_{\text{Ub}_i}$ .
435 double compute_Ub_i(int idx, double t) const {
436     double g = g_base_for(idx);
437     double M_ext, r_ext;
438     get_ext(idx, M_ext, r_ext);
439     return -beta_i * g * omega_g * (M_ext / r_ext) * U_UA
440         * std::cos(UQFF_LA002_PI * t);
441 }
442
443 /// Log $\square$  of scale range across all 8 systems (fm to Mpc).
444 double compute_scale_range() const {
445     return std::log10(r_vac / r_any); //  $\log_{10}(3.086e22 / 1e-15) \approx 37.5$ 
446 }
447
448 //  $\square$ 
449 // §B PhD-Exclusive Compute Methods
450 //  $\square$ 
451
452 /// §B.1  $\log\square|F_{\text{U}_\text{Bi}_i}|$  for the three DPM-integral systems.
453 /// Returns the stored calibrated logarithmic scale of the DPM integral.
454 double compute_DPM_log_scale(int idx) const {
455     switch (idx) {
456         case SYS_W2_TRIADIC: return 208.32; //  $\log_{10}(2.11e208)$ 
457         case SYS_SGRA_BUOY: return 211.92; //  $\log_{10}(8.31e211)$ 
458         case SYS_PILLARS_DPM: return 212.32; //  $\log_{10}(2.11e212)$ 
459         default: return std::log10(std::abs(compute_F_UBii(idx, 0.0)));
460     }
461 }
462
463 /// §B.2 Full 26-layer resonance sum  $R(t)$ .
464 ///  $R(t) = \sum\square\square\square\square R_{\{Ug1,i\}} \cdot \cos(\omega_{\{Ug1,i\}} \cdot t)$ 

```

```

465     ///  $R_{\{Ug1,i\}} = g_{base} \cdot (1+SSq) \cdot \exp(-SSq \cdot i/26)$ 
466     ///  $\omega_{\{Ug1,i\}} = (2\pi/T_{sf}) \cdot i \cdot (1+SSq)$ 
467     double compute_26D_resonance_sum(int idx, double t) const {
468         double F_base = g_base_for(idx);
469         double T_sf = (idx == SYS_RESONANCE_26D) ? T_sf_res26
470                     : (idx == SYS_W2_TRIADIC) ? T_sf_w2
471                     : (idx == SYS_PILLARS_DPM) ? T_sf_pil
472                     : T_sf_res26;
473         double ssq_loc = (idx == SYS_RESONANCE_26D) ? SSq_res26 : SSq;
474         double sum = 0.0;
475         for (int i = 1; i <= 26; ++i) {
476             double R_i = F_base * (1.0 + ssq_loc)
477                         * std::exp(-ssq_loc * i / 26.0);
478             double omega_i = UQFF_LA002_TW0_PI / T_sf * i * (1.0 + ssq_loc);
479             sum += R_i * std::cos(omega_i * t);
480         }
481         if (logging_enabled)
482             std::cout << << "[26D] R(t=<< << t << << <<) = << << <<
483         return sum;
484     }
485
486     /// §B.3 UTe2  $\delta_n$  series coefficient at layer n (1-indexed).
487     ///  $\delta_{\{n,UTe2\}} = (2\pi)^{n/6} \times \exp(-[SSq] \cdot n/26) \times (1+f_{topo}) \times \exp(-\pi)$ 
488     double compute_delta_n_UTe2(int n) const {
489         double phi_term = std::pow(UQFF_LA002_TW0_PI, static_cast<double>(n) / 6.0);
490         double decay_term = std::exp(-SSq * n / 26.0);
491         double topo_boost = 1.0 + f_topo_ute2;
492         double cosmic_exp = std::exp(-UQFF_LA002_PI); //  $t_n \rightarrow 0$  baseline
493         double result = phi_term * decay_term * topo_boost * cosmic_exp;
494         if (logging_enabled)
495             std::cout << << "[UTe2 delta=<< << n << << <<] = << << <<
496         return result;
497     }
498
499     /// §B.4 Anyon condensate  $F_{UBii}$ .
500     ///  $F_{UBii,anyons} = -F_{rel} \times (E_{any}/E_{LEP}) \times Q_{wave} \times g(r,t) \times \exp(-\delta_c^2/2\sigma^2)$ 
501     double compute_F_UBii_anyons(double g_field = -1.0) const {
502         if (g_field < 0.0) g_field = g_base_for(SYS_ANYONS_CERN);
503         double gauss = std::exp(-delta_c_loc * delta_c_loc / (2.0 * sigma_loc * sigma_loc));
504         double result = -F_rel * (E_anyons_loc / E_LEP) * Q_wave * g_field * gauss;
505         if (logging_enabled)
506             std::cout << << "[Anyons] F_UBii = << << << result << << << N (g
507         return result;
508     }
509
510     /// §B.5  $H_{res}$  nuclear shell amplitude (DC static approximation).
511     /// Full  $H_{res}$  oscillates at nuclear transition frequency; here we return
512     /// the time-averaged DC contribution:  $A_{res} + U_{dp} \cdot SC_m \cdot k_{nuc} + S_{shell}$ .
513     ///  $Z$  = atomic number,  $A$  = mass number,  $N_n$  = neutron number.
514     double compute_H_res(double Z, double A, double N_n) const {
515         // Pairing delta: even-even  $\rightarrow +0.1$ ; odd-odd  $\rightarrow -0.1$ ; mixed  $\rightarrow 0.0$ 
516         bool Z_even = (static_cast<int>(Z) % 2 == 0);
517         bool N_even = (static_cast<int>(N_n) % 2 == 0);
518         double delta_pair = (Z_even && N_even) ? 0.1 : ((!Z_even) && (!N_even)) ?
519

```

```

520         // Shell correction via proximity to magic numbers
521         constexpr int magic[7] = {2, 8, 20, 28, 50, 82, 126};
522         double S_shell = 0.0;
523         for (int m : magic) {
524             if (std::abs(static_cast<int>(Z) - m) <= 2) S_shell += 0.1;
525             if (std::abs(static_cast<int>(N_n) - m) <= 2) S_shell += 0.1;
526         }
527
528         double A_res = k_A_hres * Z * (A / A_H_ref) * (1.0 + delta_pair);
529         double k_nuc = k0_nuc * (N_n / Z) * (1.0 + delta_pair);
530         double U_dp = k0_nuc * (A * A_H_ref) / std::max(A * A * 0.09, 1.0e-30);
531         double SC_m = 1.0; // exact
532         double H_result = A_res + U_dp * SC_m * k_nuc + S_shell;
533         if (logging_enabled)
534             std::cout <<< "[H_res Z=" <<< (int)Z <<< ",A=" <<<
535                 <<< "] A_res=" <<< A_res <<< " S_shell=" <<<
536                 <<< " -> H_res=" <<< H_result <<< "\n";
537         return H_result;
538     }
539
540     /// §B.6 U_g2 buoyancy harmonics series (truncated at M_terms).
541     ///  $U_{g2} = \sum_{m=1}^M H_m \times (1 - e^{-[SSq]^m}) \times \cos(\omega_{Ug2} \times t_n)$ 
542     ///  $H_m = \sum_{k=1}^m (1/k) \times f_{Ub}$ 
543     ///  $t_n = t_{seconds} / t_{Hubble}$ 
544     double compute_Ug2_harmonics(double t_seconds, int M_terms = 26) const {
545         double t_n = t_seconds / t_Hubble;
546         double omega_ug2 = UQFF_LA002_TWO_PI / t_Hubble; // 1.44e-17 rad/s
547         double f_ub = f_Ub_cal; // 2.20e7 (calibrated)
548         double result = 0.0;
549         double H_m = 0.0;
550         for (int m = 1; m <= M_terms; ++m) {
551             H_m += f_ub / static_cast<double>(m);
552             double damp = 1.0 - std::exp(-SSq * m);
553             result += H_m * damp * std::cos(omega_ug2 * t_n);
554         }
555         if (logging_enabled)
556             std::cout <<< "[Ug2 harmonics, M=" <<< M_terms
557                 <<< ", t_n=" <<< t_n <<< "] = " <<< result;
558         return result;
559     }
560
561     /// §B.7 Vacuum Density Series (partial sum to N_terms).
562     ///  $V_{series} = \sum_{n=1}^N (1/n^2) \times [SSq]^n \rightarrow Li^{2+}([SSq]) \approx 0.570$ 
563     double compute_Vacuum_Series(int N_terms = 26) const {
564         double result = 0.0;
565         double ssq_n = SSq;
566         for (int n = 1; n <= N_terms; ++n) {
567             result += ssq_n / std::pow(static_cast<double>(n), 2.0);
568             ssq_n *= SSq;
569         }
570         if (logging_enabled)
571             std::cout <<< "[VacSeries N=" <<< N_terms <<< "] = " <<< result;
572         return result;
573     }
574

```

```

575 /// §B.8 Full U_m (single-j approximation with Heaviside correction).
576 /// U_m = ( $\mu_j/r_j$ ) × (1 - e{-γt}.cos(πt_n)) × P_SCm × E_react
577 ///      × (1 + 1013.f_H) × (1+f_quasi) × e{-[SSq]}
578 double compute_Um_full(int idx, double t) const {
579     double M    = get_M(idx);
580     double r    = get_r(idx);
581     double B_eff = get_B_eff(idx);
582     double t_n   = t / t_Hubble;
583
584     // Calibrated μ_j: Bohr magneton scale, field-enhanced
585     double mu_j = mu_B_SI * B_eff / std::max(hbar * 1.0e-12, 1.0e-50);
586     double gamma_eff = (idx == SYS_VAC_HARMONICS) ? gamma_vac_loc : gamma_d;
587
588     double exp_term = 1.0 - std::exp(-gamma_eff * t) * std::cos(UQFF_LA002_PI * t_n);
589     double P_SCm    = 1.0;
590     double E_react  = U-UA;    // reaction energy proxy
591     double f_Heaviside = 0.01;
592     double f_quasi    = 0.01;
593
594     double Um_val = (mu_j / r) * exp_term * P_SCm * E_react
595                    * (1.0 + 1.0e13 * f_Heaviside) * (1.0 + f_quasi)
596                    * std::exp(-SSq);
597     if (logging_enabled)
598         std::cout <&& " [Um sys["< && idx <&& ""] t=" <&&
599         return Um_val;
600 }
601
602 /// §B.9 Dynamic [SSq](n, t_seconds): vacuum entanglement entropy at layer n.
603 /// [SSq](n,t) = log(p_SCm/p-UA) × n × e{-(π-t_n)}
604 double compute_SSq_dynamic(int n, double t_seconds) const {
605     double t_n    = t_seconds / t_Hubble;
606
607     double log_ratio = std::log(rho_vac_SCm / rho_vac-UA);    // ≈ -2.303
608     return log_ratio * n * std::exp(-(UQFF_LA002_PI - t_n));
609 }
610
611 // ████████████████████████████████████████████████████████████████████████████████
612 // §C Print / Display Methods
613 // ████████████████████████████████████████████████████████████████████████████████
614
615 /// Print the PhD-level student guide.
616 void printPhDGuide() const {
617     std::cout <&& " \n\n";
618     std::cout <&& "=====
619     std::cout <&& " UQFF Learning Assessment 002 – PhD Research Edition (v1.0)\n\n";
620     std::cout <&& " Audience: PhD quantum physics / astrophysics / cosmology\n\n";
621     std::cout <&& "=====
622
623     std::cout <&& " FRAMEWORK OVERVIEW\n\n";
624     std::cout <&& "-----\n\n";
625     std::cout <&& " This assessment extends the Research Student Edition (_001)\n\n";
626     std::cout <&& " to 8 systems spanning 37+ log-decades of scale (1 fm -> 1 Mpc)\n\n";
627     std::cout <&& " 10 unique PhD-level physics processes are evaluated:\n\n";
628     std::cout <&& " 1. DPM 4-component buoyancy integral (F_U-Bi_i ~ 10^208..211 N/m^2)\n\n";
629     std::cout <&& " 2. 26D resonance sum R(t) = sum_{i=1}^{26} R_i cos(omega_i t)\n\n";
630     std::cout <&& " 3. UTe2 topological SC delta_n series (n=1..9, f_topo=0.20)\n\n";
631     std::cout <&& "=====
632 }

```

```

6300 std::cout <<< <<< <<< <<< 4. Anyon F_UBii: Gaussian collapse exp(-delta_c^2/2sigma^2)\n";
6301 std::cout <<< <<< <<< <<< 5. H_res nuclear shell model (Fe-56 peak, magic numbers)\n";
6302 std::cout <<< <<< <<< <<< 6. Vacuum Density Series: sum(1/n^26)[SSq]^n = Li26([SSq])\n";
6303 std::cout <<< <<< <<< <<< 7. Buoyancy Harmonics: U_g2 = sum H_m (1-e^{-[SSq]m}) cos(...\n";
6304 std::cout <<< <<< <<< <<< 8. Full Um: P_Scm x E_react x f_Heaviside x f_quasi x e^{[SSq]}\n";
6305 std::cout <<< <<< <<< <<< 9. Dynamic [SSq](n,t) = log(rho_Scm/rho_UA) x n x e^{-(pi-t_n)\n";
6306 std::cout <<< <<< <<< <<< 10. F_rel / E_LEP / Q_wave relativistic calibration (2024 LEP)\n";
6307
6308 std::cout <<< <<< <<< <<< "SYSTEM CATALOGUE\n";
6309 std::cout <<< <<< <<< <<< "-----\n";
6310 std::cout <<< <<< <<< <<< std::setw(4) <<< <<< "Idx";
6311 <<< <<< std::setw(42) <<< <<< "Name";
6312 <<< <<< std::setw(20) <<< <<< "g_base (m/s^2)";
6313 <<< <<< std::setw(24) <<< <<< "Primary PhD Method\n";
6314 std::cout <<< <<< <<< <<< std::string(90, '#39;-#39;') <<< <<< "\n";
6315 const char* methods[NUM_SYSTEMS_002] = {
6316     <<< "DPM integral log10|F|", <<< "DPM integral log10|F|", <<< "DPM integr",
6317     <<< "26D resonance sum R(t)", <<< "UTE2 delta_n series", <<< "Anyon F_U",
6318     <<< "H_res(Z,A,N)", <<< "Ug2 harmonics + VacSeries";
6319 };
6320 for (int i = 0; i << NUM_SYSTEMS_002; ++i) {
6321     std::cout <<< <<< <<< <<<
6322         <<< <<< std::setw(4) <<< <<< i
6323         <<< <<< std::setw(42) <<< <<< system_name(i)
6324         <<< <<< std::setw(20) <<< <<< std::scientific <<< <<< std::setprecision(3)
6325         <<< <<< compute_g_base(i)
6326         <<< <<< std::setw(24) <<< <<< methods[i] <<< <<< "\n";
6327 }
6328
6329 std::cout <<< <<< <<< <<< "SCALE RANGE\n";
6330 std::cout <<< <<< <<< <<< "-----\n";
6331
6332 std::cout <<< <<< <<< <<< "r_min = r_any = " <<< <<< r_any
6333 <<< <<< <<< " m [sys 5: anyon, nuclear scale]\n";
6334 std::cout <<< <<< <<< <<< "r_max = r_vac = " <<< <<< r_vac
6335 <<< <<< <<< " m [sys 7: 1 Mpc cosmic void]\n";
6336 std::cout <<< <<< <<< <<< "log10(r_max/r_min) = " <<< <<< log10(r_max/r_min) <<< <<< " std::fixed <<< <<< std::setprecision(2)
6337 <<< <<< <<< compute_scale_range() <<< <<< " decades (37.49 expected)\n";
6338
6339 std::cout <<< <<< <<< <<< "KEY PhD CONSTANTS (UQFF PhD Edition)\n";
6340 std::cout <<< <<< <<< <<< "-----\n";
6341 std::cout <<< <<< <<< <<< "F_rel = " <<< <<< F_rel <<< <<< " std::scientific <<< <<< F_rel
6342 <<< <<< <<< " N (2024 LEP calibration)\n";
6343 std::cout <<< <<< <<< <<< "E_LEP = " <<< <<< E_LEP <<< <<< " J (200 G
6344 std::cout <<< <<< <<< <<< "Q_wave = " <<< <<< Q_wave <<< <<< " (coupl
6345 std::cout <<< <<< <<< <<< "[SSq] = " <<< <<< SSq <<< <<< " (vacuum
6346 std::cout <<< <<< <<< <<< "gamma_decay = " <<< <<< gamma_d <<< <<< " s^-1 (5
6347 std::cout <<< <<< <<< <<< "delta_c_any = " <<< <<< delta_c_any <<< <<< " (
6348 std::cout <<< <<< <<< <<< "B_thr(UTE2) = " <<< <<< B_thr <<< <<< " T\n";
6349 std::cout <<< <<< <<< <<< "Li26([SSq]) = " <<< <<< compute_Vacuum_Series(100) <<< <<< "
6350
6351 std::cout <<< <<< <<< <<< "UTE2 delta_n SERIES (n=1..9)\n";
6352 std::cout <<< <<< <<< <<< "-----\n";
6353 for (int n = 1; n <<= 9; ++n)
6354     std::cout <<< <<< <<< <<< "delta_" <<< <<< n <<< <<< " = " <<< <<< compute_delta_n_UTe2(n) <<< <<< "
6355     <<< <<< <<< <<< " std::setprecision(4) <<< <<< compute_delta_n_UTe2(n) <<< <<< "

```

[illegible]


```

740     });
741
742     for (int i : order) {
743         double g    = compute_g_base(i);
744         double ubi   = compute_Ubi(i);
745         double fii   = compute_F_UBii(i, t);
746         double um    = compute_Um_full(i, t);
747         std::cout << std::left
748             << std::setw(4) << i
749             << std::setw(34) << system_name(i)
750             << std::setw(16) << std::scientific << std::setprecision(3)
751             << std::setw(16) << ubi
752             << std::setw(18) << fii
753             << std::setw(18) << um << " \n";
754     }
755     std::cout << " \n";
756     std::cout << " NOTE: F_UBii shown is Tier-2 standard (DPM-scaled for sys[0-2]) \n";
757     std::cout << " For calibrated DPM integral use compute_DPM_log_scale(idx). \n";
758     std::cout << " For anyone: compute_F_UBii_anyons(); 26D: compute_26D_resonance. \n";
759 }
760
761 /// Print compact parameter dump for all 8 systems.
762 void printParameters() const {
763     std::cout << " \n--- UQFFLearningAssessment002 Parameters --- \n";
764     std::cout << std::scientific << std::setprecision(4);
765     auto row = [&](const char* tag, double v) {
766         std::cout << " " << tag << std::left << std::setw(22) << v << " ";
767     };
768     std::cout << " [0] Westerlund 2 Triadic \n";
769     row("M_w2", M_w2); row("r_w2", r_w2);
770     row("DPM_res_w2", DPM_res_w2); row("F_LEN_R_w2", F_LEN_R_w2);
771     row("T_sf_w2", T_sf_w2);
772     std::cout << " [1] Sgr A* Buoyancy \n";
773     row("M_sgra_b", M_sgra_b); row("r_sgra_b", r_sgra_b);
774     row("DPM_res_sgra", DPM_res_sgra); row("F_LEN_R_sgra", F_LEN_R_sgra);
775     std::cout << " [2] Pillars DPM \n";
776     row("M_pil_d", M_pil_d); row("r_pil_d", r_pil_d);
777     row("DPM_res_pil", DPM_res_pil); row("F_LEN_R_pil", F_LEN_R_pil);
778     std::cout << " [3] 26D Resonance \n";
779     row("M_res26", M_res26); row("r_res26", r_res26);
780     row("N_layers", N_layers_res26); row("T_sf_res26", T_sf_res26);
781     row("SSq_res26", SSq_res26);
782     std::cout << " [4] UTe2 SC \n";
783     row("M_ute2", M_ute2); row("r_ute2", r_ute2);
784     row("B_ute2", B_ute2); row("f_topo_ute2", f_topo_ute2);
785     std::cout << " [5] Anyons CERN \n";
786     row("M_any", M_any); row("r_any", r_any);
787     row("E_anyons_loc", E_anyons_loc); row("delta_c_loc", delta_c_loc);
788     std::cout << " [6] H_res Fe-56 \n";
789     row("M_fe56", M_fe56); row("r_fe56", r_fe56);
790     row("Z_fe56", Z_fe56); row("A_fe56", A_fe56);
791     row("N_fe56", N_fe56);
792     std::cout << " [7] Vacuum Harmonics \n";
793     row("M_vac", M_vac); row("r_vac", r_vac);
794     row("N_harmonics", N_harmonics_vac); row("gamma_vac", gamma_vac_loc);

```

[illegible]

[illegible]

[illegible]


```

1015         auto it = dynamic_params.find(key);
1016         if (it != dynamic_params.end()) return it->second;
1017         if (logging_enabled)
1018             std::cout <<< <<< "[LOG] Dynamic param not found: " <<< key <<< <<< "\n";
1019         return std::numeric_limits<double>::quiet_NaN();
1020     }
1021
1022     void exportState(const std::string& filename = "UQFFLearningAssessment002_state.txt&");
1023     std::ofstream ofs(filename);
1024     if (!ofs) {
1025         std::cerr <<< "Error: Cannot open " <<< filename <<< "\n";
1026         return;
1027     }
1028     ofs <<< "# UQFFLearningAssessment002 (PhD Research Edition) state export\n";
1029     ofs <<< std::scientific <<< std::setprecision(10);
1030     const char* names[] = {
1031         // [0] W2
1032         "M_w2", "r_w2", "DPM_res_w2", "F_LEN_R_w2", "M_ext_w2", "r_ext_w2",
1033         // [1] SgrA*
1034         "M_sgra_b", "r_sgra_b", "DPM_res_sgra", "F_LEN_R_sgra", "M_ext_sgra_b", "r_ext_sgra_b",
1035         // [2] Pillars
1036         "M_pil_d", "r_pil_d", "DPM_res_pil", "F_LEN_R_pil", "M_ext_pil_d", "r_ext_pil_d",
1037         // [3] 26D
1038         "M_res26", "r_res26", "N_layers_res26", "T_sf_res26", "M_ext_res26", "r_ext_res26",
1039         // [4] UTe2
1040         "M_ute2", "r_ute2", "B_ute2", "f_topo_ute2", "M_ext_ute2", "r_ext_ute2",
1041
1042         // [5] Anyons
1043         "M_any", "r_any", "E_anyons_loc", "delta_c_loc", "M_ext_any", "r_ext_any",
1044         // [6] Fe-56
1045         "M_fe56", "r_fe56", "Z_fe56", "A_fe56", "M_ext_fe56", "r_ext_fe56",
1046         // [7] Vac
1047         "M_vac", "r_vac", "N_harmonics_vac", "gamma_vac_loc", "M_ext_vac", "r_ext_vac",
1048         // metrics
1049         "diversity_score", "dynamic_score", "scalability_score",
1050     };
1051     for (const char* n : names)
1052         ofs <<< n <<< " = " <<< getVariable(n) <<< "\n";
1053     for (const auto& kv : dynamic_params)
1054         ofs <<< "dynamic." <<< kv.first <<< " = " <<< kv.second <<< "\n";
1055     if (logging_enabled)
1056         std::cout <<< "[LOG] State exported to: " <<< filename <<< "\n";
1057 }
1058
1059 /// Cross-validate g_base[sys_idx] against an external UQFF module.
1060 /// Store the other module's g beforehand: setDynamicParameter("cross_g", val).
1061 template<typename OtherModuleT>
1062 double cross_validate(const OtherModuleT& other, int sys_idx,

```

```

1070             double /*t_seconds*/ = 0.0) const {
1071     double g_here = compute_g_base(sys_idx);
1072     double g_other = 0.0;
1073     auto it = dynamic_params.find("cross_g");
1074     if (it != dynamic_params.end()) g_other = it->second;
1075     double frac = (g_here != 0.0)
1076         ? std::abs(g_other - g_here) / std::abs(g_here)
1077         : std::numeric_limits<double>::quiet_NaN();
1078     if (logging_enabled)
1079         std::cout << " [XVAL002] sys[" << sys_idx << " ] g_here="
1080             << g_here << " g_other=" << g_other
1081             << " frac_diff=" << frac << " \n";
1082     return frac;
1083 }
1084 };
1085
1086 #endif // UQFF_LEARNING_ASSESSMENT_002_H

```

— End of UQFFLearningAssessment_002.cpp —